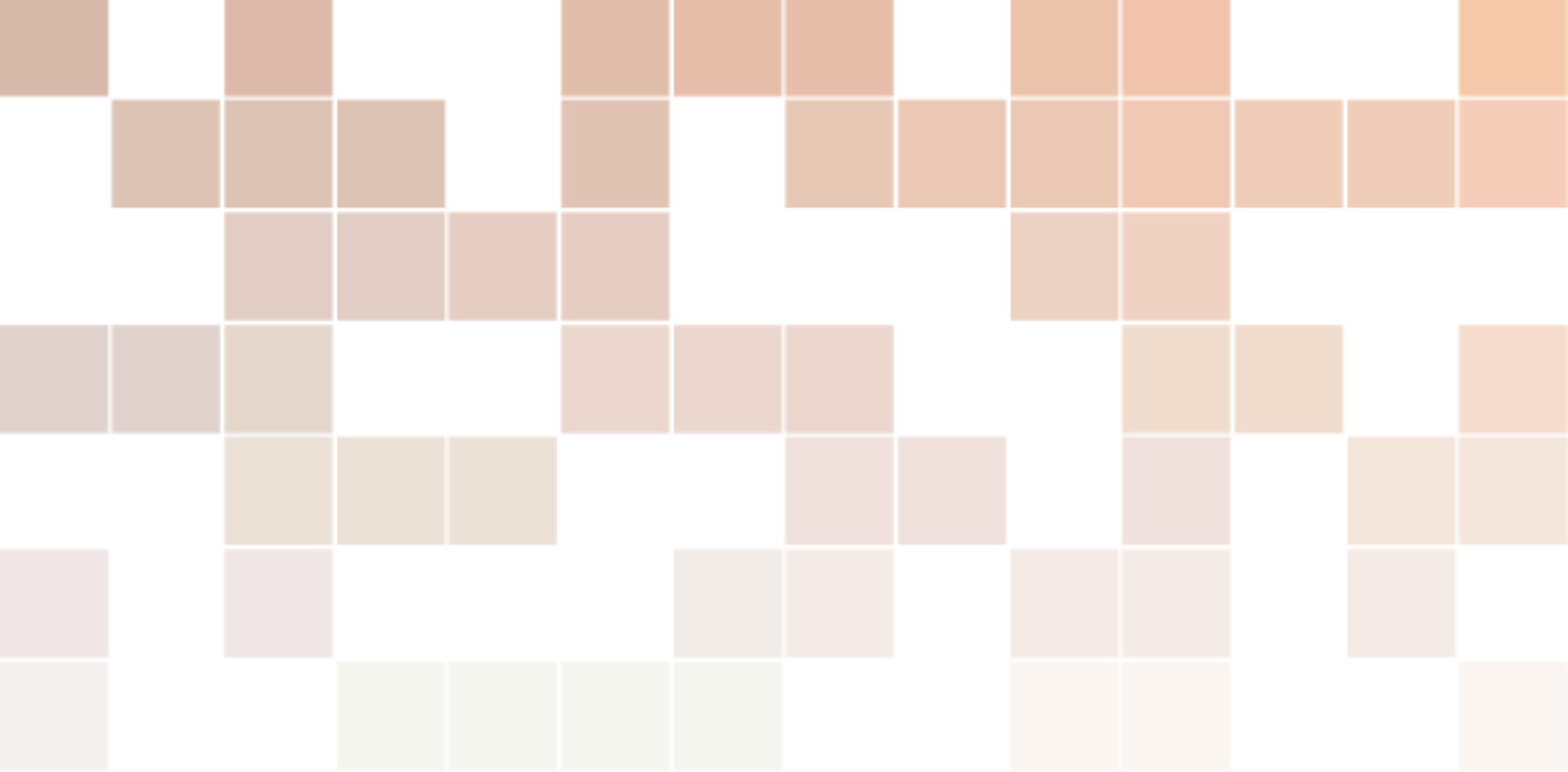
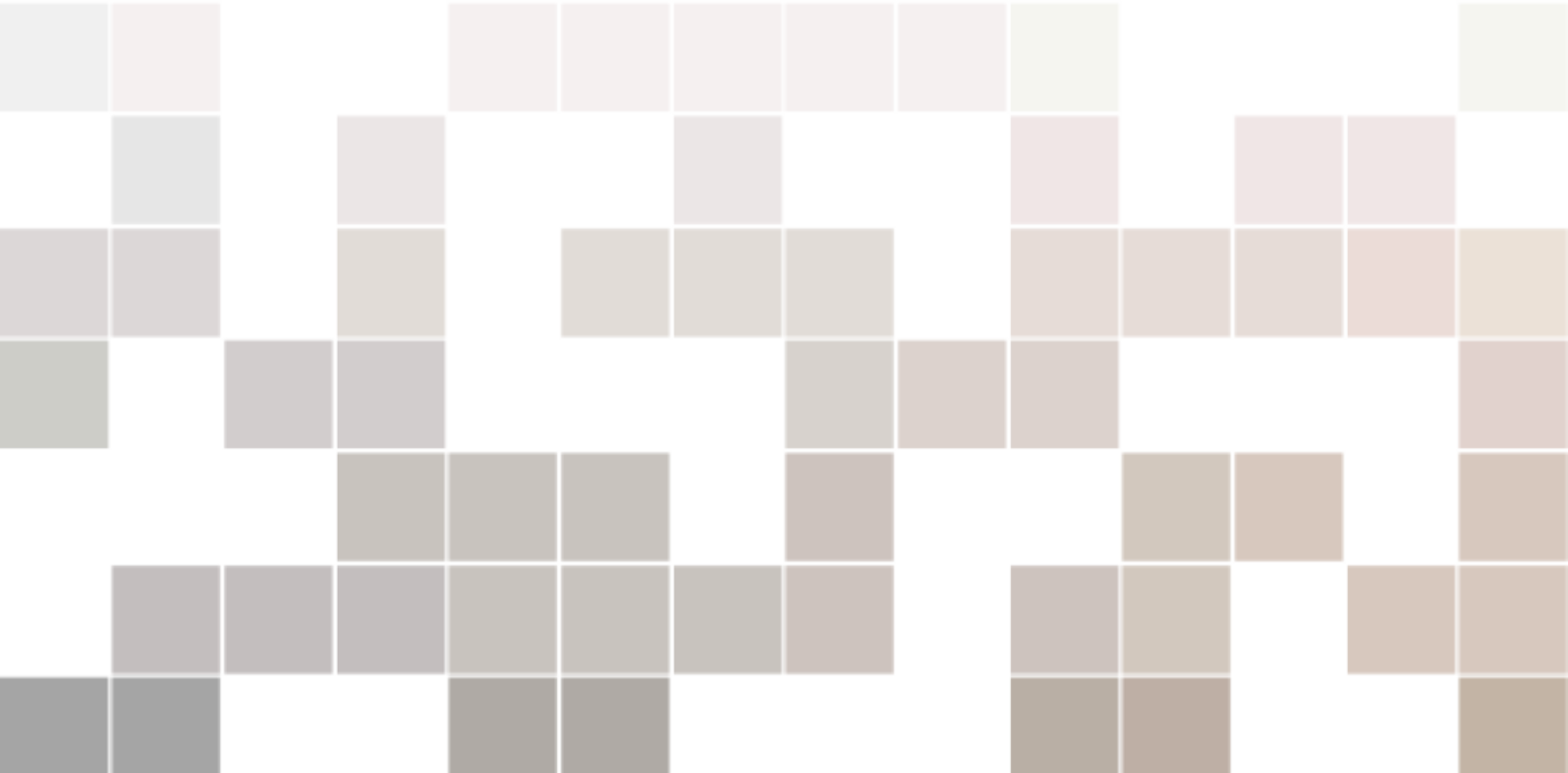




机器人学教程

理论与实践

左元





目录

I

现代机器人状态估计

1	概论	11
---	----------	----

II

Point-LIO 算法

2	源码解读	15
---	------------	----

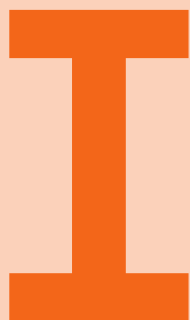


List of Figures

2.1	23
-----------	----



List of Tables



现代机器人状态估计

1	概论	11
----------	---------------------	-----------



1. 概论

🔥 机器人的状态估计要解决的问题

我的机器人现在在哪里？

I

I

Point-LIO 算法

2. 源码解读

```
1  #ifndef S03_MATH_H
2  #define S03_MATH_H
3
4  #include <math.h>
5  #include <Eigen/Core>
6
7  // #include <common_lib.h>
8
9  #define SKEW_SYM_MATRX(v) 0.0,-v[2],v[1],v[2],0.0,-v[0],-v[1],v[0],0.0
10
11 template<typename T>
12 Eigen::Matrix<T, 3, 3> skew_sym_mat(const Eigen::Matrix<T, 3, 1> &v)
13 {
14     Eigen::Matrix<T, 3, 3> skew_sym_mat;
15     skew_sym_mat<<0.0,-v[2],v[1],v[2],0.0,-v[0],-v[1],v[0],0.0;
16     return skew_sym_mat;
17 }
18
19 template<typename T>
20 Eigen::Matrix<T, 3, 3> Exp(const Eigen::Matrix<T, 3, 1> &ang)
21 {
22     T ang_norm = ang.norm();
23     Eigen::Matrix<T, 3, 3> Eye3 = Eigen::Matrix<T, 3, 3>::Identity();
24     if (ang_norm > 0.0000001)
25     {
26         Eigen::Matrix<T, 3, 1> r_axis = ang / ang_norm;
27         Eigen::Matrix<T, 3, 3> K;
28         K << SKEW_SYM_MATRX(r_axis);
29         /// Roderigious Tranformation
30         return Eye3 + std::sin(ang_norm) * K + (1.0 - std::cos(ang_norm)) * K * K;
31     }
32     else
33     {
```



```

34     return Eye3;
35 }
36 }
37
38 template<typename T, typename Ts>
39 Eigen::Matrix<T, 3, 3> Exp(const Eigen::Matrix<T, 3, 1> &ang_vel, const Ts &dt)
40 {
41     T ang_vel_norm = ang_vel.norm();
42     Eigen::Matrix<T, 3, 3> Eye3 = Eigen::Matrix<T, 3, 3>::Identity();
43
44     if (ang_vel_norm > 0.0000001)
45     {
46         Eigen::Matrix<T, 3, 1> r_axis = ang_vel / ang_vel_norm;
47         Eigen::Matrix<T, 3, 3> K;
48
49         K << SKEW_SYM_MATRX(r_axis);
50
51         T r_ang = ang_vel_norm * dt;
52
53         /// Roderigious Tranformation
54         return Eye3 + std::sin(r_ang) * K + (1.0 - std::cos(r_ang)) * K * K;
55     }
56     else
57     {
58         return Eye3;
59     }
60 }
61
62 template<typename T>
63 Eigen::Matrix<T, 3, 3> Exp(const T &v1, const T &v2, const T &v3)
64 {
65     T &&norm = sqrt(v1 * v1 + v2 * v2 + v3 * v3);
66     Eigen::Matrix<T, 3, 3> Eye3 = Eigen::Matrix<T, 3, 3>::Identity();
67     if (norm > 0.00001)
68     {
69         T r_ang[3] = {v1 / norm, v2 / norm, v3 / norm};
70         Eigen::Matrix<T, 3, 3> K;
71         K << SKEW_SYM_MATRX(r_ang);
72
73         /// Roderigious Tranformation
74         return Eye3 + std::sin(norm) * K + (1.0 - std::cos(norm)) * K * K;
75     }
76     else
77     {
78         return Eye3;
79     }
80 }
81
82 /* Logrithm of a Rotation Matrix */
83 template<typename T>

```

```

84 Eigen::Matrix<T,3,1> Log(const Eigen::Matrix<T, 3, 3> R)
85 {
86     T theta = (R.trace() > 3.0 - 1e-6) ? 0.0 : std::acos(0.5 * (R.trace() - 1));
87     Eigen::Matrix<T,3,1> K(R(2,1) - R(1,2), R(0,2) - R(2,0), R(1,0) - R(0,1));
88     return (std::abs(theta) < 0.001) ? (0.5 * K) : (0.5 * theta / std::sin(theta)
      * K);
89 }
90
91 template<typename T>
92 Eigen::Matrix<T, 3, 1> RotMtoEuler(const Eigen::Matrix<T, 3, 3> &rot)
93 {
94     T sy = sqrt(rot(0,0)*rot(0,0) + rot(1,0)*rot(1,0));
95     bool singular = sy < 1e-6;
96     T x, y, z;
97     if(!singular)
98     {
99         x = atan2(rot(2, 1), rot(2, 2));
100        y = atan2(-rot(2, 0), sy);
101        z = atan2(rot(1, 0), rot(0, 0));
102    }
103    else
104    {
105        x = atan2(-rot(1, 2), rot(1, 1));
106        y = atan2(-rot(2, 0), sy);
107        z = 0;
108    }
109    Eigen::Matrix<T, 3, 1> ang(x, y, z);
110    return ang;
111 }
112
113 template<typename T>
114 Eigen::Matrix3d Jacob_right_inv(Eigen::Vector3d &vec){
115     Eigen::Matrix3d hat_v, res;
116     hat_v << SKEW_SYM_MATRX(vec);
117     if(vec.norm() > 1e-6)
118     {
119         res = Eigen::Matrix<double, 3, 3>::Identity() + 0.5 * hat_v + (1 -
      vec.norm() * std::cos(vec.norm() / 2) / 2 / std::sin(vec.norm() / 2)) *
      hat_v * hat_v / vec.squaredNorm();
120     }
121     else
122     {
123         res = Eigen::Matrix<double, 3, 3>::Identity();
124     }
125     return res;
126 }
127
128 #endif

```

S03_MATH 代码详解

整体定位

```
1 S0(3) = Special Orthogonal Group (特殊正交群)
2       = 所有 3×3 旋转矩阵的集合
3
4 这个头文件提供了 S0(3) 上的核心数学工具:
5   旋转向量 ↔ 旋转矩阵 的相互转换
6   用于 Point-LIO 的状态估计
```

一、基础概念

旋转的三种表示

```
1 旋转向量 (轴角):  $\omega = \theta \cdot \hat{n}$ 
2    $\theta$  = 旋转角度 (弧度)
3    $\hat{n}$  = 旋转轴 (单位向量)
4   向量的模 = 旋转角度
5   向量的方向 = 旋转轴
6
7 旋转矩阵 R: 3×3 矩阵,  $R^T \cdot R = I$ ,  $\det(R) = 1$ 
8
9 欧拉角: roll, pitch, yaw (有万向锁问题)
```

李群与李代数

```
1 S0(3) = 李群 (旋转矩阵的集合)
2 so(3) = 李代数 (反对称矩阵的集合)
3
4 Exp() : so(3) → S0(3) (李代数 → 李群)
5 Log() : S0(3) → so(3) (李群 → 李代数)
6
7 直觉:
8   旋转矩阵不能直接相加 (不在线性空间)
9   但旋转向量可以相加
10  Exp/Log 就是在这两个空间之间转换的桥梁
```

二、skew_sym_mat — 反对称矩阵 (叉积矩阵)

```
1 template<typename T>
2 Eigen::Matrix<T, 3, 3> skew_sym_mat(const Eigen::Matrix<T, 3, 1> &v)
3 {
4     skew_sym_mat << 0.0, -v[2], v[1],
5                     v[2], 0.0, -v[0],
6                     -v[1], v[0], 0.0;
7 }
```

数学定义

$$\hat{v} = \begin{pmatrix} 0 & -v_z & v_y \\ v_z & 0 & -v_x \\ -v_y & v_x & 0 \end{pmatrix}$$

(2.1)

核心性质

$$\hat{v} \cdot u = v \times u \quad (\text{叉积})$$

(2.2)

- 1 把向量叉积转换成矩阵乘法
- 2 这是 Rodrigues 公式的基础

验证

$$v = (1, 2, 3), u = (4, 5, 6) \quad (2.3)$$

$$v \times u = \begin{vmatrix} i & j & k \\ 1 & 2 & 3 \\ 4 & 5 & 6 \end{vmatrix} = (-3, 6, -3) \quad (2.4)$$

$$\hat{v} \cdot u = \begin{pmatrix} 0 & -3 & 2 \\ 3 & 0 & -1 \\ -2 & 1 & 0 \end{pmatrix} \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} = \begin{pmatrix} -3 \\ 6 \\ -3 \end{pmatrix} \quad (2.5)$$

三、Exp — 旋转向量转旋转矩阵 Rodrigues 旋转公式

$$R = \exp(\hat{\omega}) = I + \sin \theta \cdot K + (1 - \cos \theta) \cdot K^2 \quad (2.6)$$

其中 $K = \hat{n}$ (旋转轴的反对称矩阵), $\theta = \|\omega\|$

代码实现 (版本 1: 输入旋转向量)

```
1  template<typename T> C++
2  Eigen::Matrix<T, 3, 3> Exp(const Eigen::Matrix<T, 3, 1> &ang)
3  {
4      T ang_norm = ang.norm();          //  $\theta = |\omega|$  (旋转角度)
5
6      if (ang_norm > 0.0000001)
7      {
8          Eigen::Matrix<T, 3, 1> r_axis = ang / ang_norm; //  $\hat{n} = \omega/\theta$  (单位旋转轴)
9          Eigen::Matrix<T, 3, 3> K;
10         K << SKEW_SYM_MATRX(r_axis); // K =  $\hat{n}$  的反对称矩阵
11
12         // Rodrigues 公式
13         return Eye3 + std::sin(ang_norm) * K + (1.0 - std::cos(ang_norm)) * K * K;
14     }
15     else
16     {
17         return Eye3; // 旋转角度 $\approx 0$ , 返回单位矩阵
18     }
19 }
```

代码实现 (版本 2: 输入角速度 + 时间步长)

```
1  template<typename T, typename Ts> C++
2  Eigen::Matrix<T, 3, 3> Exp(const Eigen::Matrix<T, 3, 1> &ang_vel, const Ts &dt)
3  {
4      // 旋转角度 = 角速度 * 时间
5      T r_ang = ang_vel.norm() * dt; //  $\theta = |\omega| \times dt$ 
6
7      // 同样用 Rodrigues 公式
8      return Eye3 + std::sin(r_ang) * K + (1.0 - std::cos(r_ang)) * K * K;
9  }
```

```

1 用途: IMU 积分
2   已知角速度  $\omega$  (rad/s) 和时间间隔  $dt$ 
3   计算这段时间内的旋转矩阵
4    $R_{new} = R_{old} \times \text{Exp}(\omega, dt)$ 

```

Rodrigues 公式几何直觉

```

1 绕轴  $\hat{n}$  旋转  $\theta$  角:
2
3  $R = I$  → 不旋转的部分 (投影到旋转轴)
4  $+ \sin(\theta) \cdot K$  → 旋转90°的分量
5  $+ (1 - \cos(\theta)) \cdot K^2$  → 旋转180°的分量
6
7 三项叠加 = 完整的旋转

```

为什么小角度返回单位矩阵

```

1 当  $\theta \rightarrow 0$  时:
2    $\sin(\theta) \rightarrow \theta \rightarrow 0$ 
3    $1 - \cos(\theta) \rightarrow \theta^2/2 \rightarrow 0$ 
4
5 整个旋转趋近于  $I$ 
6 直接返回  $I$ , 避免除以接近0的数导致数值不稳定

```

四、Log — 旋转矩阵转旋转向量

```

1  template<typename T> C++
2  Eigen::Matrix<T,3,1> Log(const Eigen::Matrix<T, 3, 3> R)
3  {
4      // 从迹 (trace) 提取旋转角度
5      // trace(R) = 1 + 2cos(θ)
6      T theta = (R.trace() > 3.0 - 1e-6) ? 0.0
7          : std::acos(0.5 * (R.trace() - 1));
8
9      // 从反对称部分提取旋转轴方向
10     //  $R - R^T = 2\sin(\theta) \cdot K \rightarrow K$  的元素在这里
11     Eigen::Matrix<T,3,1> K(R(2,1) - R(1,2), // 2sin(θ) · n_x
12         R(0,2) - R(2,0), // 2sin(θ) · n_y
13         R(1,0) - R(0,1)); // 2sin(θ) · n_z
14
15     // 小角度: 直接用  $0.5 \cdot K$  (因为  $\sin(\theta) \approx \theta$ , 所以  $K \approx 2\theta \cdot \hat{n}$ ,  $0.5K \approx \theta \cdot \hat{n}$ )
16     // 正常角度:  $\theta / (2\sin(\theta)) \times K = \theta \cdot \hat{n}$  (旋转向量)
17     return (std::abs(theta) < 0.001) ? (0.5 * K)
18         : (0.5 * theta / std::sin(theta) * K);
19 }

```

数学推导

$$\text{trace}(R) = 1 + 2 \cos \theta \Rightarrow \theta = \arccos\left(\frac{\text{trace}(R) - 1}{2}\right) \quad (2.7)$$

$$R - R^T = 2 \sin \theta \cdot K \Rightarrow \text{旋转向量} = \frac{\theta}{2 \sin \theta} (R - R^T)^V \quad (2.8)$$

其中 $(\cdot)^v$ 表示从反对称矩阵提取向量

五、RotMtoEuler — 旋转矩阵转欧拉角

```
1  template<typename T>
2  Eigen::Matrix<T, 3, 1> RotMtoEuler(const Eigen::Matrix<T, 3, 3> &rot)
3  {
4      // sy = cos(pitch), 用于判断是否奇异
5      T sy = sqrt(rot(0,0)*rot(0,0) + rot(1,0)*rot(1,0));
6      bool singular = sy < 1e-6; // pitch = ±90° 时奇异 (万向锁)
7
8      if(!singular)
9      {
10         x = atan2(rot(2,1), rot(2,2)); // roll
11         y = atan2(-rot(2,0), sy);      // pitch
12         z = atan2(rot(1,0), rot(0,0)); // yaw
13     }
14     else // 万向锁情况
15     {
16         x = atan2(-rot(1,2), rot(1,1)); // roll (特殊处理)
17         y = atan2(-rot(2,0), sy);      // pitch = ±90°
18         z = 0;                        // yaw 无法确定, 置0
19     }
20 }
```

ZYX 旋转矩阵结构

$$R_{ZYX} = \begin{pmatrix} c_y c_p & c_y s_p s_r - s_y c_r & c_y s_p c_r + s_y s_r \\ s_y c_p & s_y s_p s_r + c_y c_r & s_y s_p c_r - c_y s_r \\ -s_p & c_p s_r & c_p c_r \end{pmatrix} \quad (2.9)$$

$$\begin{aligned} c_y &= \cos(\text{yaw}) \\ s_y &= \sin(\text{yaw}) \\ c_p &= \cos(\text{pitch}) \\ s_p &= \sin(\text{pitch}) \\ c_r &= \cos(\text{roll}) \\ s_r &= \sin(\text{roll}) \end{aligned} \quad (2.10)$$

从矩阵元素反推:

$$\begin{aligned} \text{rot}(2,0) &= -\sin(\text{pitch}) \rightarrow \text{pitch} = \text{atan2}(-\text{rot}(2,0), s_y) \\ \frac{\text{rot}(2,1)}{\text{rot}(2,2)} &= \frac{\sin(\text{roll})}{\cos(\text{roll})} \rightarrow \text{roll} \\ \frac{\text{rot}(1,0)}{\text{rot}(0,0)} &= \frac{\sin(\text{yaw})}{\cos(\text{yaw})} \rightarrow \text{yaw} \end{aligned} \quad (2.11)$$

六、Jacob_right_inv — 右雅可比逆矩阵

```
1  template<typename T>
2  Eigen::Matrix3d Jacob_right_inv(Eigen::Vector3d &vec)
3  {
4      Eigen::Matrix3d hat_v;
5      hat_v << SKEW_SYM_MATRX(vec);
```

```
6
7     if(vec.norm() > 1e-6)
8     {
9         res = I + 0.5 * hat_v
10            + (1 -  $\theta \cdot \cos(\theta/2)$  / ( $2 \cdot \sin(\theta/2)$ )) * hat_v^2 /  $\theta^2$ 
11     }
12     else
13     {
14         res = I; // 小角度近似
15     }
16 }
```

用途

在 $SO(3)$ 上做优化时，需要把李代数的扰动映射回切空间（Tangent Space）

J_r^{-1} 用于：

- 残差对旋转的雅可比矩阵计算
- Point-LIO 中的迭代卡尔曼滤波更新步骤

直觉：

- 旋转空间是弯曲的（非线性）
- J_r^{-1} 是这个弯曲空间的“局部线性化修正”

七、完整关系图

旋转向量 ω (轴角)

$$\|\omega\| = \text{旋转角度}$$

$$\frac{\omega}{\|\omega\|} = \text{旋转轴}$$



$$\exp(\omega) = I + \sin(\theta)K + (1 - \cos(\theta))K^2$$

旋转矩阵 $R \in SO(3)$

$$R^T \cdot R = I$$

$$\det(R) = 1$$



$$\log R$$

旋转向量 ω (还原)



$$\text{RotMtoEuler}(R)$$

欧拉角(roll, pitch, yaw)

- 有万向锁问题
- 不适合插值

图 2.1

八、在 Point-LIO 中的使用
IMU 预积分:

$$R_{t+dt} = R_t \times \exp(\omega_{\text{imu}} \times dt) \quad (2.12)$$

角速度积分得到旋转增量

状态更新:

$\partial R = \exp(\partial \varphi)$ 将误差状态 (李代数) 转为旋转矩阵

残差计算:

$r = \log(R_{\text{pred}}^T \times R_{\text{meas}})$ 两个旋转矩阵的“差”

雅可比计算:

$\frac{\partial r}{\partial \varphi} = J_r^{-1}$ 用于迭代卡尔曼滤波的线性化